

Arquitectura Integral & Documentación del Frontend

Manual exhaustivo sobre tecnologías, componentes, arquitectura cliente-servidor, flujos de datos reactivos y suite de pruebas del Ecosistema RIWI HSE Barranquilla.

PROYECTO

Email Automation & HSE Attendance System

VERSIÓN FRONTEND

2.4.0 (React 19 + Tailwind v4 + Vite 8)

ESTADO OPERATIVO

19 / 19 Pruebas E2E Aprobadas (100% Pasando)

AUDIENCIA

Desarrolladores, QA Engineers, Arquitectos & HSE

Resumen de Conformidad de Calidad (QA)

El sistema cuenta con suite completa de pruebas E2E en Playwright, arquitectura desacoplada tolerante a fallos de red con fallback de semillas, tipado estricto TypeScript y diseño responsivo móvil/tablet/escritorio sin desbordamientos.

1. Resumen Ejecutivo y Arquitectura General

El **Frontend del Ecosistema RIWI HSE** es una aplicación web moderna de una sola página (Single Page Application - SPA) diseñada para administrar de manera integral el control de ausentismos, radicación de justificaciones médicas/personales, análisis de datos de salud ocupacional y auditoría diaria de asistencia para más de 290 coders en la sede RIWI Barranquilla.

La aplicación se articula en torno a dos portales principales que conviven bajo el mismo núcleo reactivo pero con accesos estrictamente segmentados mediante control de acceso basado en roles (RBAC):

Portal HSE / Team Leader (TL)

Bandeja de solicitudes de inasistencia, revisión de evidencias médicas PDF, decisión de convalidación apoyada en IA, gestión de parámetros de tolerancia y directorio con métricas consolidadas por ruta formativa.

Portal del Coder (Estudiante)

Radicación en 2 pasos de justificaciones con soporte probatorio, consulta en tiempo real del estado de sus solicitudes ("Convalidada / Aprobada"), visualización de "Mi Asistencia" y centro de comunicados "Chat HSE".

Topología y Flujo de Comunicación

El frontend opera como un cliente enriquecido alojado en Vite. Su estrategia de comunicación con el backend (FastAPI / Uvicorn en puerto `8001`) se fundamenta en un **enfoque híbrido resiliente**:

1 Proxy Inverso de Desarrollo (Vite Proxy)

Todas las llamadas a `/api/*` son retransmitidas por Vite de forma transparente hacia el backend en `http://localhost:8001`, eliminando problemas de CORS durante desarrollo y centralizando la configuración.

2 Tolerancia a Desconexión y Modo Offline Hermético

El cliente integra capas de intercepción en `api.ts`. Si el backend se encuentra detenido o la red falla (`ECONNREFUSED`), el sistema recurre inmediatamente a semillas canónicas (`mockStudents`, `mockStats`), evitando pantallas blancas y asegurando que las pruebas automatizadas y demos operen sin depender de bases de datos externas.

3 Sincronización Reactiva Bidireccional basada en Eventos DOM

Cuando un Coder radica una nueva excusa, el cliente emite eventos nativos `hse_justification_created` y `hse_justifications_updated` vía `window.dispatchEvent`. La bandeja de solicitudes de la TL se suscribe a estos eventos, actualizándose instantáneamente sin requerir recargar la página (F5).

2. Stack Tecnológico Completo y Herramientas

La aplicación está construida sobre las versiones más recientes y estables del ecosistema moderno de JavaScript/TypeScript, priorizando velocidad de compilación, rendimiento de renderizado y seguridad estática de tipos:

TECNOLOGÍA / LIBRERÍA	VERSIÓN	CATEGORÍA	PROPÓSITO Y RAZÓN DE ELECCIÓN
React	^19.2.8	Core Framework	Motor de UI declarativo. Implementa concurrencia, carga asíncrona optimizada y hooks avanzados (<code>useMemo</code> , <code>useCallback</code>).
TypeScript	~6.0.2	Lenguaje	Tipado estricto de extremo a extremo para modelos canónicos (<code>Student</code> , <code>Request</code> , <code>CoderJustification</code>).
Vite	^8.3.0	Bundler & Tooling	Servidor de desarrollo instantáneo con Hot Module Replacement (HMR) ultrarrápido y construcción en producción con optimización Rollup.
Tailwind CSS	^4.3.3	Diseño & Estilos	Nuevo motor nativo con compilación Lightning CSS. Cero configuración, variables <code>@theme</code> , diseño responsivo estricto (móvil, tablet, escritorio).
React Router DOM	^7.18.4	Enrutamiento SPA	Manejo de rutas declarativas protegidas por RBAC, redirecciones de seguridad y layouts compartidos sin recargas.
Framer Motion	^13.4.4	Animaciones UI	Transiciones suaves, renderizado progresivo de listas, modales animados y drawer de navegación móvil colapsable.
React Hook Form	^7.89.0	Manejo de Formularios	Captura eficiente del formulario de radicación sin re-renderizados innecesarios del DOM.
Zod + Resolvers	^4.6.5 / ^5.9	Validación Estricta	Esquemas de validación para motivos, fechas de ausencia y términos legales.
React Dropzone	^20.1.2	Subida de Archivos	Área de arrastrar y soltar con validación estricta de extensiones (PDF, PNG, JPG) y tamaño de archivos de evidencia.
Recharts	^3.10.1	Visualización de Datos	Gráficas vectoriales interactivas para tendencias semanales y métricas de ausentismo en Dashboard y Reportes.
Lucide React	^1.48.0	Iconografía	Iconos vectoriales ligeros y consistentes en toda la aplicación.
Playwright Test	^1.63.0	Testing E2E & QA	Suite automatizada de 19 pruebas de integración y comportamiento del usuario ejecutadas con Chromium headless.

TECNOLOGÍA / LIBRERÍA	VERSIÓN	CATEGORÍA	PROPÓSITO Y RAZÓN DE ELECCIÓN
Oxlint	^1.81.0	Análisis Estático	Linter de alto rendimiento en Rust para detección rápida de errores de sintaxis y buenas prácticas.

3. Catálogo Detallado de Archivos del Frontend

A continuación se detalla la estructura completa de carpetas del proyecto y la responsabilidad específica asignada a cada archivo dentro de la solución:

```
frontend/src/
├─ App.tsx           // Enrutador principal, Providers y Definición de Rutas RBAC
├─ main.tsx          // Punto de entrada SPA, montaje en DOM (div#root)
├─ index.css          // Importación de Tailwind CSS v4 (@import "tailwindcss")
├─ pages/             // Vistas principales vinculadas a rutas URL
│  ├─ Dashboard.tsx   // Panel HSE: 4 KPIs globales, métricas semanales y resumen
│  ├─ Requests.tsx    // Bandeja de solicitudes TL, visor de PDF y exportar reporte
│  ├─ Students.tsx    // Directorio de coders, filtro de rutas y mensaje masivo
│  ├─ Reports.tsx     // Reportes analíticos de ausentismo y distribución mensual
│  ├─ Settings.tsx    // Configuración interactiva de reglas y tolerancias HSE
│  ├─ Login.tsx       // Pantalla de acceso responsiva con credenciales de prueba
│  └─ coder/
│     ├─ ExcuseSubmissionForm.tsx // Formulario Coder de 2 pasos con carga de evidencia
│     └─ CoderHistory.tsx        // Historial de radicados, "Mi Asistencia" y "Chat HSE"
├─ components/        // Componentes reutilizables de UI y navegación
│  ├─ Sidebar.tsx     // Menú lateral HSE colapsable con botón hamburguesa (3 rayas)
│  ├─ Header.tsx      // Barra superior con notificaciones y perfil de usuario
│  ├─ MobileHeader.tsx // Barra móvil con trigger para drawer lateral
│  └─ coder/
│     ├─ CoderSidebar.tsx // Navegación lateral específica del portal Coder
│     └─ CoderProfileMenu.tsx // Menú flotante de perfil con logout no invasivo
├─ students/
│  └─ CoderAttendanceHistory.tsx // 4 tarjetas dinámicas y modal de edición de asistencia
├─ services/          // Capas de servicio y comunicación HTTP
│  ├─ api.ts           // Cliente de API con fallbacks offline y sincronización
│  └─ hseMessages.ts   // Despacho de comunicados masivos y mensajes individuales
├─ context/           // Estado global reactivo
│  └─ AuthContext.tsx  // Autenticación, persistencia de token JWT y control de sesión
├─ utils/             // Funciones auxiliares y adaptadores de dominio
│  ├─ jwt.ts           // Decodificación Base64URL de tokens y normalización de roles
│  ├─ coderJustifications.ts // Adaptador de justificaciones Coder a Requests de la TL
│  ├─ coderSession.ts  // Gestión de datos de sesión activa en localStorage
│  └─ textUtils.ts     // Búsqueda tolerante a tildes y mayúsculas/minúsculas
├─ data/              // Semillas y datos de respaldo mock
│  └─ mock.ts          // Lista de coders, KPIs y solicitudes para fallback offline
├─ types/             // Declaraciones de tipos e interfaces
│  └─ index.ts         // Modelos Student, Request, Attendance, CoderJustification
```

4. Fichas Técnicas de Páginas y Componentes

Í Módulo HSE & Team Leader

`src/pages/Dashboard.tsx` — Panel de Control Principal

Es la vista central del analista HSE. Presenta 4 tarjetas métricas dinámicas: *Total Solicitudes*, *Aprobadas*, *Denegadas* y *Por Revisar*. Incluye el componente gráfico `Recharts` que mapea la distribución semanal de radicaciones y un acceso directo a la bandeja con el botón "Ver Solicitudes".

`src/pages/Requests.tsx` — Bandeja de Solicitudes y Resolución

Diseñada bajo el patrón arquitectónico **Maestro-Detalle**. En la columna izquierda lista las justificaciones con filtros de carpetas (Bandeja, Convalidadas, Denegadas, Por Revisar). Al seleccionar una solicitud, la columna derecha despliega el detalle completo:

- Información del Coder, cédula, ruta y motivo declarado.
- **Visor de Evidencias:** Enlace interactivo al PDF adjunto con previsualización `Ver evidencia ↗`.
- **Consola de Decisión:** Campos para notas de auditoría y botones de acción (*Aprobar Excusa*, *Denegar* o *Pedir Corrección*).
- **Botón "Descargar reporte":** Ubicado en la esquina superior derecha, genera una descarga CSV con codificación UTF-8 y Byte Order Mark (BOM) para compatibilidad nativa con Excel sin dañar caracteres acentuados.

`src/pages/Students.tsx` — Directorio de Coders y Comunicados

Gestiona el censo de estudiantes agrupados dinámicamente por rutas académicas (*Desarrollo Web Fullstack*, *Node.js Backend*, *Automatización con IA*, etc.). Incorpora:

- Buscador reactivo tolerante a acentos mediante `matchesNormalized`.
- **Botón "Mensaje a la Ruta":** Abre un modal modal con opciones de alcance (Ruta actual, Todas las rutas o Selección manual múltiple de rutas) para emitir comunicados masivos oficiales.
- Al hacer clic sobre cualquier tarjeta de coder (`[data-testid="coder-card"]`), transiciona fluidamente hacia el historial individual de asistencia.

`src/components/students/CoderAttendanceHistory.tsx` — Historial y Modificación Diaria

Componente de auditoría detallada de un Coder específico:

- **4 Cuadros Dinámicos:** Contadores de *Asistencia*, *Retraso*, *Faltas Justificadas* y *Faltas Injustificadas* calculados de forma coherente.
- **Bitácora Cronológica:** Detalle día por día con iconos de estado, hora de llegada y motivos de inasistencia.
- **Botón Modificar Asistencia:** Despliega un modal interactivo con 4 tarjetas de selección (Presente, Retraso con llegada posterior, Falta justificada con soporte, Falta injustificada no soportada) que actualiza la asistencia en tiempo real.
- **Botón "Enviar Mensaje al Coder":** Permite emitir observaciones y citaciones personalizadas dirigidas exclusivamente al buzón del estudiante.

5. Portal del Coder, Sincronización y Seguridad

Módulo Coder

src/pages/coder/ExcuseSubmissionForm.tsx — Formulario de Radicación por Pasos

Permite al estudiante radicar justificaciones siguiendo un proceso estructurado en 2 fases:

- **Paso 1 (Motivo & Fechas):** Selección del tipo de novedad (Incapacidad Médica EPS, Calamidad Doméstica, Trámite Legal, Falla Técnica), rango de fechas de ausencia, explicación detallada y declaración juramentada de veracidad.
- **Paso 2 (Carga de Evidencias):** Integración con `React Dropzone` para anexar soportes PDF o imágenes, verificación de legibilidad y vista previa inmediata.
- **Confirmación y Radicado:** Emisión automática de código único de seguimiento con formato `RAD-HSE-YYYY-XXXX`.

src/pages/coder/CoderHistory.tsx — Portal Personal del Coder

Punto de encuentro del estudiante con 3 pestañas o secciones principales:

- **Historial de Solicitudes:** Muestra el estado de sus trámites con insignias a color (*Convalidada / Aprobada* en verde, *Rechazada* en rojo, *En Revisión* en amarillo) y las notas emitidas por la TL.
- **Mi Asistencia:** Permite consultar sus 4 cuadros métricos de asistencia, retrasos y faltas de manera idéntica a la vista HSE, pero en modo de solo lectura (sin permisos de edición).
- **Chat HSE:** Bandeja interna donde el estudiante recibe los comunicados oficiales masivos de su ruta y los mensajes personales enviados por bienestar.
- **Menú de Perfil (`CoderProfileMenu.tsx`):** Avatar en la esquina superior derecha con botón "Cerrar sesión" que limpia de forma segura el almacenamiento local y redirige al login sin errores.

Seguridad y Control de Acceso (RBAC)

La aplicación implementa un sistema estricto de rutas protegidas mediante `ProtectedRoute` en `App.tsx`:

ROL NORMALIZADO	RUTAS PERMITIDAS	RESTRICCIONES Y REDIRECCIONES
<code>HSE_ANALYST</code> / <code>ADMIN</code>	<code>/</code> , <code>/requests</code> , <code>/students</code> , <code>/reports</code> , <code>/settings</code>	Acceso total a funciones administrativas, convalidación de excusas y gestión de directrices.
<code>CODER</code>	<code>/coder/new-excuse</code> , <code>/coder/history</code>	Si intenta ingresar a rutas administrativas (<code>/</code> o <code>/requests</code>), el sistema lo intercepta automáticamente y lo redirige a <code>/coder/new-excuse</code> .
<code>ANÓNIMO</code>	<code>/login</code>	Cualquier intento de navegación a rutas autenticadas redirige de inmediato a <code>/login</code> .

6. Suite de Pruebas Automatizadas (Playwright E2E)

El frontend cuenta con una suite completa de **19 pruebas automatizadas End-to-End** implementadas con Playwright. Todas las pruebas se ejecutan de manera hermética con un tiempo total de aproximadamente 1 minuto:

ARCHIVO DE TEST	PRUEBAS	ASPECTOS VALIDADOS
<code>coder-to-tl-sync.spec.ts</code>	1 test	Sincronización bidireccional completa: radicación de excusa con PDF por Coder -> aparición instantánea en bandeja TL -> aprobación formal por TL -> verificación en historial del Coder.
<code>hse-coder-new-features.spec.ts</code>	7 tests	Validación de botón "Descargar reporte" en Requests, eliminación de preferencias en Settings, mensaje masivo a rutas y edición de asistencia en 4 tarjetas, sección Chat HSE, sección Mi Asistencia, sidebar con hamburguesa de 3 rayas y logout de perfil Coder.
<code>coder-flow.spec.ts</code>	1 test	Flujo integral del Coder: login, llenado de formulario por pasos, drag & drop de evidencia PDF y obtención de número de radicado.
<code>hse-flow.spec.ts</code>	1 test	Flujo del analista HSE: login administrativo, apertura de bandeja, revisión de solicitud y resolución manual.
<code>rbac-guard.spec.ts</code>	2 tests	Protección de rutas: bloqueo de usuarios no autenticados y aislamiento estricto de roles Coder fuera del panel HSE.
<code>responsive-*.spec.ts</code>	6 tests	Pruebas de diseño responsivo en iPhone SE (375px) y iPad (768px): verificación de ausencia de scroll horizontal (overflow) en Login, Dashboard, Requests, Students, Layout y Formulario Coder.
<code>smoke.spec.ts</code>	1 test	Smoke test de carga inicial del login, tipografías y renderizado de la mascota astronáutica de RIWI.

Garantía de Calidad en Integración Continua (CI)

Gracias a la interceptación hermética de `/api/students` en `auth.ts` y los fallbacks de `api.ts`, los tests nunca sufren bloqueos por `ECONNREFUSED` y pasan al 100% de manera determinista y rápida.

Comandos Clave de Operación

- `npm run dev`: Inicia el servidor de desarrollo Vite en `http://localhost:5173` con proxy hacia FastAPI.
- `npm run build`: Realiza la comprobación estricta de TypeScript (`tsc -b`) y compila el bundle de producción optimizado con Vite en `/dist` (~2 segundos).
- `npm run test:e2e`: Ejecuta las 19 pruebas Playwright en modo headless reportando el estado en consola.
- `npm run preview`: Previsualiza el bundle compilado de producción localmente.

Fin del Documento Técnico Oficial • RIWI HSE Barranquilla • Documentación generada para fines de arquitectura, auditoría y desarrollo continuo.